
Ayudantía de Preparación – Prueba N°2

Ayudante: Matías Fuentes

Esta ayudantía tiene como objetivo repasar los contenidos evaluados en la Prueba N°2: **estructuras iterativas** (while, do-while, for, simples o anidadas), **funciones** (prototipo, definición, llamada y traspaso de parámetros por valor), **ámbito de variables** (global y local), además de un repaso de la materia de la evaluación anterior (tipos de variables, sintaxis, if/else y switch).

1. ¿Qué ciclo utilizarías?

Un cajero automático simplificado debe solicitar al usuario que ingrese su PIN (un número entero de 4 dígitos). El sistema debe volver a solicitar el PIN **las veces que sea necesario** mientras el valor ingresado no tenga exactamente 4 dígitos (es decir, mientras sea menor a 1000 o mayor a 9999), ya que la primera solicitud siempre debe realizarse sin importar si el usuario ya conoce su PIN o no.

- a) Proponga y justifique brevemente **cuál de las tres estructuras iterativas** (while, do-while o for) es la más adecuada para este escenario, considerando cuántas veces como mínimo se debe ejecutar la solicitud del PIN.
- b) Implemente el programa completo en C utilizando la estructura que usted propuso, mostrando el mensaje "PIN valido" una vez que el usuario ingrese un valor correcto.

2. Encuentre y corrija los errores

El siguiente fragmento de código pretende calcular el promedio de N notas ingresadas por el usuario, y clasificar dicho promedio en "Aprobado" (mayor o igual a 4.0) o "Reprobado". Sin embargo, el código contiene **4 errores** (de sintaxis, de lógica o de buenas prácticas) que impiden su correcto funcionamiento.

```
1 #include <stdio.h>
2
3 int main() {
4     int n;
5     float nota, suma;
6     float promedio;
7
8     printf("Cuántas notas ingresará?: ");
9     scanf("%d", &n);
10
11     for (i = 1; i <= n; i++) {
12         printf("Ingrese nota %d: ", i);
13         scanf("%f", &nota);
14         suma = suma + nota;
15     }
16
17     promedio = suma / n;
18     printf("El promedio es: %.2f\n", promedio);
19
20     if (promedio >= 4.0)
21         printf("Aprobado\n");
22     else
23         printf("Reprobado\n");
24
25     return 0;
26 }
```

Identifique cada error señalando el número de línea, explique brevemente por qué es un error, y reescriba el código completo ya corregido.

3. Funciones en cadena

- a) Defina el prototipo y la implementación de una función llamada `esPrimo`, que reciba un número entero como parámetro y retorne un valor de tipo `int` que actúe como booleano (1 si el número es primo, 0 si no lo es). Considere que los números menores a 2 no son primos.
- b) Utilizando la función `esPrimo` definida en (a), implemente una segunda función llamada `contarPrimosEnRango`, que reciba dos números enteros `inicio` y `fin` como parámetros, y retorne (tipo `int`) la cantidad de números primos que existen en dicho rango (incluyendo ambos extremos).
- c) Escriba una función `main` que solicite al usuario dos valores enteros, los use para invocar `contarPrimosEnRango` y muestre por pantalla la cantidad de números primos encontrados.

Hint: Para verificar si un número es primo, basta con comprobar que no tenga divisores exactos entre 2 y su raíz (o, de forma más simple para este curso, entre 2 y el número menos 1).

4. ¿Por qué no funciona como se espera?

El siguiente programa fue diseñado para **duplicar** el valor de una variable ingresada por el usuario, utilizando una función llamada `duplicar`. Al compilarlo y ejecutarlo, el programa **no genera ningún error** y termina correctamente, pero el valor mostrado en pantalla **nunca cambia** respecto al valor ingresado originalmente.

```
1 #include <stdio.h>
2
3 void duplicar(int numero) {
4     numero = numero * 2;
5 }
6
7 int main() {
8     int valor;
9
10    printf("Ingrese un numero: ");
11    scanf("%d", &valor);
12
13    duplicar(valor);
14
15    printf("El valor duplicado es: %d\n", valor);
16
17    return 0;
18 }
```

- a) Explique, en sus propias palabras, **por qué** el programa no produce el resultado esperado, haciendo referencia explícita a la forma en que C traspasa los parámetros a las funciones.
- b) Proponga una **corrección** al código (puede modificar la función, la forma en que se invoca, o ambas) de manera que el valor mostrado en pantalla sí corresponda al valor original duplicado.

5. Implemente el código solicitado

Escriba un programa en C que, utilizando **ciclos anidados**, imprima en pantalla un patrón triangular de asteriscos como el siguiente, para un valor de N ingresado por el usuario (en el ejemplo, $N = 5$):

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Adicionalmente, el programa debe declarar una variable **global** llamada `totalAsteriscos` que vaya acumulando la cantidad total de asteriscos impresos a lo largo de todo el patrón, y mostrarla en pantalla una vez finalizado el dibujo.

Hint: Recuerde que una variable declarada fuera de cualquier función tiene ámbito **global**, por lo que puede ser modificada directamente desde dentro de un ciclo en `main` sin necesidad de pasarla como parámetro.